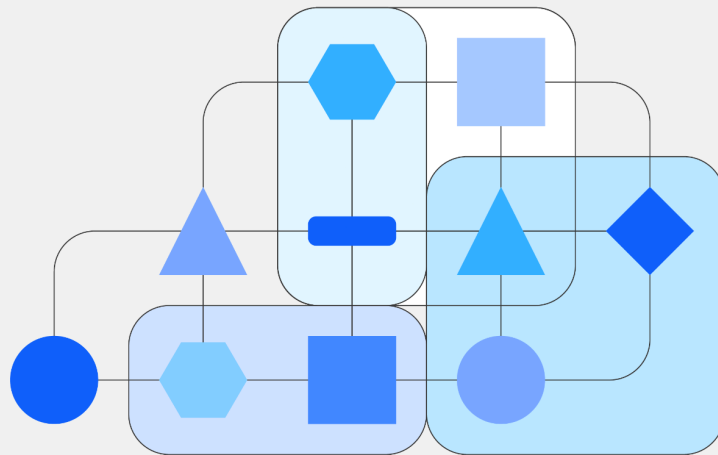


ACTIVIDAD 3

Estructura de datos

Docente: Alberto Lemus Mera



ÍNDICE

Objetivo.....	1
Instrucciones.....	1
Código.....	1
Secuencia Fibonacci.....	1
Subconjunto en un Conjunto que suma un Objetivo.....	2
Resolver Sudoku.....	3
Código Corriendo.....	5
Secuencia Fibonacci.....	5
Subconjunto en un Conjunto que suma un Objetivo.....	5
Resolver Sudoku.....	6

Objetivo

- Aplicar conceptos de recursividad directa e indirecta.
- Implementar algoritmos de backtracking y divide y vencerás.
- Desarrollar la habilidad para resolver problemas complejos mediante la descomposición en subproblemas.

Instrucciones

1. **Primer problema: Serie de Fibonacci recursiva.** Implementa una función recursiva en Java que calcule el enésimo número en la serie de Fibonacci. Asegúrate de incluir tanto el caso base como el caso recursivo.
2. **Segundo problema: Suma de subconjuntos (Subset Sum).** Desarrolla un algoritmo recursivo para determinar si existe un subconjunto de un conjunto dado de enteros que sume un valor objetivo.
3. **Tercer problema: Algoritmo de backtracking para el problema del sudoku.** Implementa un algoritmo de backtracking que resuelva un Sudoku. El programa debe llenar las celdas vacías del tablero de Sudoku dado.

Código

Secuencia Fibonacci

```
public static int secuenciaFibonacci(int n) {  
    // Caso Base  
    if (n == 0) {  
        return 0;  
    } else if (n == 1) {  
        return 1;  
    }  
  
    // Recursividad  
    return secuenciaFibonacci(n - 1) + secuenciaFibonacci(n - 2);  
}  
  
public static void main(String[] args) throws Exception{  
    System.out.println("Enesimo número de la secuencia fibonacci: " +  
secuenciaFibonacci(11));  
}
```

Subconjunto en un Conjunto que suma un Objetivo

```
public static boolean subset_sum(int [] conjunto, int n, int objetivo) {  
    // Caso Base  
    if (objetivo == 0) {  
        return true;  
    } else if (n == 0) {  
        return false;  
    }  
  
    // Recursividad  
    if (conjunto[n-1] > objetivo) {  
        return subset_sum(conjunto, n-1, objetivo);  
    }  
  
    return subset_sum(conjunto, n-1, objetivo) ||  
subset_sum(conjunto, n-1, objetivo - conjunto[n-1]);  
  
}  
  
public static void main(String[] args) throws Exception{  
    int[] conjunto = {3, 34, 4, 12, 5, 2};  
    int objetivo = 9;  
    int n = conjunto.length;  
    System.out.println("Existe un subconjunto en el conjunto: ");  
    for (int i = 0; i < conjunto.length; i++)  
        System.out.println(conjunto[i]);  
    System.out.println(" que suma: " + objetivo);  
    if (subset_sum(conjunto, n, objetivo)) {  
        System.out.println("Si hay un subconjunto que suma: " +  
objetivo);  
    } else {  
        System.out.println("No hay un subconjunto que suma: " +  
objetivo);  
    }  
}
```

Resolver Sudoku

```
private static int[] encontrarVacia(int[][] tablero) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (tablero[i][j] == 0) {
                return new int[]{i, j};
            }
        }
    }
    return null;
}

private static boolean esValido(int[][] tablero, int num, int fila,
int col) {
    for (int j = 0; j < 9; j++) {
        if (tablero[fila][j] == num) {
            return false;
        }
    }

    for (int i = 0; i < 9; i++) {
        if (tablero[i][col] == num) {
            return false;
        }
    }

    int cajaFila = fila - fila % 3;
    int cajaCol = col - col % 3;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (tablero[cajaFila + i][cajaCol + j] == num) {
                return false;
            }
        }
    }

    return true;
}
```

```
public static boolean resolverSudoku(int[][] tablero) {
    int[] vacia = encontrarVacia(tablero);

    // Caso Base
    if (vacía == null) {
        return true;
    }

    // Recursividad
    int fila = vacia[0];
    int col = vacia[1];

    for (int num = 1; num <= 9; num++) {
        if (esValido(tablero, num, fila, col)) {
            tablero[fila][col] = num;

            if (resolverSudoku(tablero)) {
                return true;
            }

            tablero[fila][col] = 0;
        }
    }

    return false;
}

public static void imprimirTablero(int[][] tablero) {
    for (int r = 0; r < 9; r++) {
        for (int d = 0; d < 9; d++) {
            System.out.print(tablero[r][d] + " ");
        }
        System.out.print("\n");
    }
}

public static void main(String[] args) throws Exception{
    System.out.println("--- Sudoku ---");
    int[][] tablero = {
        {5, 3, 0, 0, 7, 0, 0, 0, 0},
    }
```

```
{6, 0, 0, 1, 9, 5, 0, 0, 0},
{0, 9, 8, 0, 0, 0, 0, 6, 0},
{8, 0, 0, 0, 6, 0, 0, 0, 3},
{4, 0, 0, 8, 0, 3, 0, 0, 1},
{7, 0, 0, 0, 2, 0, 0, 0, 6},
{0, 6, 0, 0, 0, 0, 2, 8, 0},
{0, 0, 0, 4, 1, 9, 0, 0, 5},
{0, 0, 0, 0, 8, 0, 0, 7, 9}

};

System.out.println("--- Tablero Inicial ---");
imprimirTablero(tablero);

if (resolverSudoku(tablero)) {
    System.out.println("\n--- Sudoku Resuelto ---");
    imprimirTablero(tablero);
} else {
    System.out.println("\nEl Sudoku no tiene solución.");
}
}
```

Código Corriendo

Secuencia Fibonacci

```
> java App
Enesimo número de la secuencia fibonacci: 89
```

Subconjunto en un Conjunto que suma un Objetivo

```
Existe un subconjunto en el conjunto:
3
34
4
12
5
2
que sume: 9
Si hay un subconjunto que sume: 9
```

Resolver Sudoku

```
Sudoku
--- Tablero Inicial ---
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9

--- ¡Sudoku Resuelto con Éxito! ---
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```